

Production Orders API — API Specification Design (API仕様設計)

001_DD-API — supports 001_DD; handlers designed in 001_DD-FN; requirements REQ-010–REQ-018.

Document control (改版履歴)

Field	Value
Document ID	001_DD-API
System name	ProductionManagementAI
Subsystem name	Production orders
Work item	WI-002
Created by	Claude (for ThanhTN)
Created date	2026-09-18
Last updated by	Claude (for ThanhTN)
Last updated date	2026-09-22

Version	Date	Author	Revision content
1	2026-09-18	Claude (for ThanhTN)	Initial creation
2	2026-09-22	Claude (for ThanhTN)	Reviewed for WI-004 REQ-033: no contract change. The completion time is not added to <code>ProductionOrderResponse</code> (Screen A does not show it) and is not accepted in any request; the request DTOs have no such property, so a client-sent field of that name binds to nothing
3	2026-09-23	Claude (for ThanhTN)	WI-005: the UI is Japanese. The example product name is a Japanese automobile part (WI-005 DEC-002). Problem Details <code>title</code> texts stay English: they are API text for developers, never shown in the UI, which renders the message ID from <code>code</code> . No field, rule, API or processing changes

Overview and operation catalog

Field	Value
API / module name	Production orders API

Field	Value
Overview	JSON endpoints behind SCR-001. <code>GET /api/products</code> is expected to be reused by Screen B's filters, which is why this is a separate document

No	Endpoint / method	Handler	Purpose	Notes
1	<code>GET /api/products</code>	<code>ProductsController.List</code> → <code>ProductionOrderService.ListProductsAsync</code>	Product options	
2	<code>POST /api/production-orders</code>	<code>ProductionOrdersController.Create</code> → <code>CreateAsync</code>	Create an order	
3	<code>GET /api/production-orders/{id}</code>	<code>ProductionOrdersController.Get</code> → <code>GetAsync</code>	Load an order	
4	<code>PUT /api/production-orders/{id}</code>	<code>ProductionOrdersController.Update</code> → <code>UpdateAsync</code>	Update an order	Full replacement of the editable fields

Common to every endpoint:

- **Auth:** cookie session (0002_ADR) plus policy `ProductionOrderEditor` = role `Admin` or `Operator` (DEC-001). No session → 401 with an empty body. Session without the role → 403 with an empty body. This is the existing WI-001 cookie-event behavior, kept unchanged.
- **Content type:** request bodies must be `application/json`, otherwise 415 (DEC-020). Responses are `application/json`; errors are `application/problem+json` (RFC 9457).
- **JSON:** camelCase; dates as `YYYY-MM-DD`; timestamps ISO 8601 UTC with `Z`; enums as strings.
- **Error body:**

```
{
  "type": "urn:pmi:problem:validation",
  "title": "One or more fields are invalid.",
  "status": 400,
  "code": "VALIDATION",
  "errors": { "quantity": ["MSG-E003"], "dueDate": ["MSG-E005"] },
  "traceId": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01"
}
```

`errors` values are message IDs from 001_DD's catalog; the client maps them to text, so wording lives in one place. Stack traces and exception messages are never included. JSON binding failures (e.g. `"quantity": "abc"`, a malformed date) are mapped to the same shape by a custom `InvalidModelStateResponseFactory`: framework keys like `$.quantity` become `quantity`, and the framework messages become the field's message ID. Framework text is never returned.

System-wide API registry entries

ID	Endpoint	Overview	Notes
API-PRD-01	GET /api/products	List products	
API-PO-01	POST /api/production-orders	Create production order	
API-PO-02	GET /api/production-orders/{id}	Get production order	
API-PO-03	PUT /api/production-orders/{id}	Update production order	

No project-level API registry exists yet; these IDs start it.

Shared response object: ProductionOrderResponse

No	Name	Variable name	Type	Length	Required	Repeats (array)	Value mapping	Example	Description	Notes
1	ID	id	string (uuid)	36	yes	no	—	3f2c...	production_orders.id	
2	Order number	orderNumber	string	13	yes	no	—	P0-2026-00042	Generated column	
3	Product ID	productId	string (uuid)	36	yes	no	—			
4	Quantity	quantity	integer	—	yes	no	—	250		
5	Due date	dueDate	string (date)	10	yes	no	—	2026-10-01		
6	Status	status	string	—	yes	no	Enum name	InProgress		
7	Allowed next statuses	allowedNextStatuses	string[]	—	yes	yes	From status.AllowedNext()	["Completed", "Cancelled"]	Empty for terminal statuses	Drives M-02
8	Product/quantity editable	isProductQuantityEditable	boolean	—	yes	no	status == Draft	false		Drives the lock UI (REQ-018)
9	Notes	notes	string null	500	yes	no	—	null		

No	Name	Variable name	Type	Length	Required	Repeats (array)	Value mapping	Example	Description	Notes
10	Created at	<code>createdAt</code>	string (date-time)	—	yes	no	UTC	<code>2026-09-18T01:02:03Z</code>		
11	Updated at	<code>updatedAt</code>	string (date-time)	—	yes	no	UTC			
12	Version	<code>version</code>	integer (uint32)	—	yes	no	<code>xmin</code>	<code>81234</code>	Echo back on PUT	DEC-014

Endpoint / method design

1. GET /api/products

Field	Value
Description	All products, ordered by <code>sku</code> ascending
Return type	<code>ProductResponse[]</code>
Created by / date	Claude / 2026-09-18
Last modified by / date	—

Arguments: none.

Processing overview: no-tracking projection `SELECT id, sku, name FROM products ORDER BY sku`. No paging, since the list is 30 rows (DEC-019). Add paging or search when Screen B needs it.

Processing flow

Step	Description	Calls
1	Authorize	policy <code>ProductionOrderEditor</code>
2	Query and project	<code>IProductionOrderRepository.ListProductsAsync</code>
3	200	—

Return value

Type	Name	Description
<code>ProductResponse[]</code>	body	See response fields

Request fields: none.

Response fields (each array element)

No	Name	Variable name	Type	Length	Required	Repeats (array)	Value mapping	Example	Description	Notes
1	ID	<code>id</code>	string (uuid)	36	yes	yes	—			
2	SKU	<code>sku</code>	string	50	yes	yes	—	<code>P-1004</code>		
3	Name	<code>name</code>	string	200	yes	yes	—	<code>ドライブシャフト</code>		

Error codes

Code	Meaning	HTTP status
—	Not signed in	401
—	Missing role	403
<code>MSG-E013</code>	Unexpected failure	500

2. POST /api/production-orders

Field	Value
Description	Create a production order in <code>Draft</code> with the next <code>PO-YYYY-NNNNN</code> number
Return type	<code>ProductionOrderResponse</code>
Created by / date	Claude / 2026-09-18
Last modified by / date	—

Arguments: request body below.

Processing overview: server side of P-02. Method-level design is in 001_DD-FN §3 `CreateAsync` ; the controller boundary is in 001_DD-SPD §5.

Processing flow

Step	Description	Calls
1	Authorize; require the JSON content type	policy; [Consumes]
2	Required/shape/range checks; collect all field errors	ProductionOrder.ValidateQuantity / ValidateNotes
3	Product exists; <code>dueDate ≥ IPlantClock.Today</code>	repository; IPlantClock
4	Transaction: <code>IOrderNumberIssuer.NextAsync(IPlantClock.CurrentYear)</code> → <code>ProductionOrder.Create</code> → insert → commit	001_DB counter upsert
5	201, Location: <code>/api/production-orders/{id}</code>	—

Return value

Type	Name	Description
ProductionOrderResponse	body	The created order

Request fields

No	Name	Variable name	Type	Length	Required	Source	Example	Description	Notes
1	Product ID	productId	string (uuid)	36	yes	request body		Must exist	MSG-E001 / MSG-E002
2	Quantity	quantity	integer	—	yes	request body	250	1–999,999,999	MSG-E003 / MSG-E010. A non-integer JSON value (e.g. 2.5 , "abc") → MSG-E003
3	Due date	dueDate	string (date)	10	yes	request body	2026-10-01	≥ plant today	MSG-E004 / MSG-E005
4	Notes	notes	string null	500	no	request body		Trimmed; empty → null; ≤ 500 code points	MSG-E006

Other properties (e.g. `status` , `orderNumber`) are ignored; a new order is always `Draft` .

Response fields: `ProductionOrderResponse` (above).

Error codes

Code	Meaning	HTTP status
VALIDATION	One or more field errors (<code>errors</code> map)	400
—	Not signed in / missing role	401 / 403
—	Body not <code>application/json</code>	415
MSG-E013	Counter overflow or unexpected failure	500

3. GET /api/production-orders/{id}

Field	Value
Description	One order by ID
Return type	<code>ProductionOrderResponse</code>
Created by / date	Claude / 2026-09-18
Last modified by / date	—

Arguments

No	Type	Name	Description
1	uuid (route, <code>{id:guid}</code>)	<code>id</code>	A non-GUID value doesn't match the route → 404

Processing overview: no-tracking load and projection, including `xmin` as `version` .

Processing flow

Step	Description	Calls
1	Authorize	policy
2	Load	repository
3	200, or 404 <code>MSG-E011</code>	—

Return value

Type	Name	Description
ProductionOrderResponse	body	

Request fields: route `id` only.

Response fields: `ProductionOrderResponse` .

Error codes

Code	Meaning	HTTP status
MSG-E011	No order with this ID	404
—	Not signed in / missing role	401 / 403

4. PUT /api/production-orders/{id}

Field	Value
Description	Replace the editable fields and optionally change the status, guarded by <code>version</code>
Return type	<code>ProductionOrderResponse</code>
Created by / date	Claude / 2026-09-18
Last modified by / date	—

Arguments

No	Type	Name	Description
1	uuid (route)	<code>id</code>	

Processing overview: server side of P-03, method-level design in 001_DD-FN §4 `updateAsync` . The check order is fixed: 400 (shape) → 404 → 409 (version) → 422 (Domain rules) → 400 (data-dependent) → 409 (race at save).

Processing flow

Step	Description	Calls
1	Authorize; JSON content type	policy; <code>[Consumes]</code>

Step	Description	Calls
2	Shape/required/range checks	Domain validators
3	Load tracked entity → 404	repository
4	<code>version</code> $\neq$ <code>current xmin</code> → 409 MSG-E009	—
5	<code>order.Update(...)</code> → 422 with <code>code</code> MSG-E008 or MSG-E007	Domain
6	Changed product exists? Changed due date $\geq$ plant today? → 400	repository; <code>IPlantClock</code>
7	<code>SaveChanges ; DbUpdateConcurrencyException</code> → 409	EF Core
8	200 with a fresh <code>ProductionOrderResponse</code>	—

Return value

Type	Name	Description
<code>ProductionOrderResponse</code>	body	Saved state, new <code>version</code>

Request fields

No	Name	Variable name	Type	Length	Required	Source	Example	Description	Notes
1	Product ID	<code>productId</code>	string (uuid)	36	yes	request body		Must equal the saved value unless <code>Draft</code>	V-01, V-07
2	Quantity	<code>quantity</code>	integer	—	yes	request body		Same as POST; must equal the saved value unless <code>Draft</code>	V-02, V-07
3	Due date	<code>dueDate</code>	string (date)	10	yes	request body		$\geq$ plant today only if changed	V-03, V-04
4	Status	<code>status</code>	string	—	yes	request body	<code>Completed</code>	Current status or one of <code>allowedNextStatuses</code> ; unknown value → 400	V-06
5	Notes	<code>notes</code>	string null	500	no	request body		Same as POST	V-05
6	Version	<code>version</code>	integer (uint32)	—	yes	request body	<code>81234</code>	From the last GET/PUT response	V-08

Response fields: `ProductionOrderResponse` .

Error codes

Code	Meaning	HTTP status
<code>VALIDATION</code>	Field errors (<code>errors</code> map)	400

Code	Meaning	HTTP status
—	Not signed in / missing role	401 / 403
MSG-E011	No such order	404
MSG-E009	Stale version	409
—	Body not application/json	415
MSG-E007 / MSG-E008	Disallowed transition / locked field changed	422
MSG-E013	Unexpected failure	500

Problem type URIs: urn:pmai:problem:validation, ...:not-found, ...:conflict, ...:rule-violation, ...:internal. The top-level code is VALIDATION for 400 (details in errors) and the 001_DD message ID for every other error (404 MSG-E011, 409 MSG-E009, 422 MSG-E007 / MSG-E008, 500 MSG-E013). 401/403/415 keep their framework bodies.

Unresolved decisions

None. Error-contract choices are recorded as DEC-023 in work-items/WI-002/decisions.md.